

Razor Pages con .NET 10

Manual completo para aprender a construir aplicaciones web, consumir API, trabajar con bases de datos y publicar proyectos mantenibles

Finalidad

Pensado para alumnado que ya conoce los fundamentos de C# y quiere comprender el recorrido completo de una petición web: desde el navegador y el enrutamiento hasta el PageModel, la base de datos, las API y la respuesta HTML.

Al terminar este manual podrás:

- Crear, ejecutar, depurar y publicar una aplicación Razor Pages dirigida a **net10.0**.
- Separar correctamente interfaz, lógica de página, servicios, modelos, DTO y acceso a datos.
- Implementar CRUD asíncrono con EF Core 10 y SQLite, relaciones, búsqueda, ordenación y paginación.
- Consumir API externas de forma robusta con **IHttpClientFactory**, JSON, caché y gestión de errores.
- Exponer una API propia, habilitar CORS cuando sea necesario y crear un cliente web sencillo.
- Integrar Bootstrap, Bootstrap Icons, Bootswatch, SweetAlert2 y JavaScript pequeño y accesible.
- Aplicar seguridad, pruebas, diagnóstico, rendimiento y criterios de calidad adecuados.

Edición didáctica 2026 · .NET 10

Índice

1. Mapa del manual y de los proyectos	6
Familias de problemas cubiertas	6
Ruta de aprendizaje recomendada	6
2. Web, HTTP y arquitectura de ASP.NET Core	7
Petición y respuesta	7
De la URL a la página	7
Razor Pages, MVC, API y Blazor	7
3. Crear y conocer un proyecto .NET 10	8
SDK y comandos esenciales	8
El archivo de proyecto	8
Estructura mínima	9
Actividades propuestas	9
4. Program.cs, servicios y middleware	10
Configuración mínima	10
Ciclos de vida de la inyección de dependencias	10
Registrar servicios propios	10
5. Razor: HTML dinámico sin perder claridad	12
Directivas principales	12
Codificación automática	12
Bloques de código y funciones	12
6. PageModel, handlers y resultados	13
Una pareja de archivos	13
Convención de handlers	13
Resultados habituales	13

7. Routing, parámetros y generación de enlaces	15
Convención de carpetas	15
Route values y query string	15
Tag Helpers para no escribir URL a mano	15
8. Model binding, formularios y validación	16
Modelo de entrada separado	16
ModelState	16
9. Interfaz compartida: layout, parciales y estado efímero	18
_Layout.cshtml	18
_ViewImports y _ViewStart	18
ViewData, ViewBag y TempData	18
10. Configuración, secretos y logging	19
Configuración por capas	19
Options pattern	19
Logging estructurado	20
11. Entity Framework Core 10 y SQLite	21
Paquetes y herramientas	21
Entidad y DbContext	21
Migraciones	21
Seguimiento y consultas de lectura	22
12. CRUD asíncrono y relaciones	23
Listar con proyección	23
Crear	23
Editar sin sobreasignación	23
Borrar y controlar relaciones	24
Carga de datos relacionados	24

13. LINQ, búsqueda, normalización y paginación	25
Consulta componible	25
Búsqueda sin tildes ni diferencias de mayúsculas	25
Modelo de paginación	26
Actividades propuestas	26
14. Programación asíncrona en aplicaciones web	27
Por qué async/await	27
Buenas prácticas	27
15. Consumir API externas con HttpClient y DTO	28
Cliente tipado	28
DTO: un contrato pequeño	28
Estados de interfaz	29
IMemoryCache	29
16. API propia, controladores, JSON y CORS	30
Razor Pages y controladores en el mismo proyecto	30
CORS	30
Cliente estático	31
17. SignalR y estado en tiempo real	32
Cuándo usarlo	32
Diseño de mensajes	32
18. JavaScript pequeño: debounce, temas y almacenamiento local	33
Debounce de búsqueda	33
Tema persistente	33
dataset	34
Evitar lógica duplicada	34
19. Bootstrap, iconos, Bootswatch y SweetAlert2	35

Dependencias locales con LibMan	35
---	----

Confirmación de borrado reutilizable	35
--	----

20. Seguridad: antifalsificación, Identity y autorización **37**

Protecciones por defecto	37
------------------------------------	----

Autenticación y autorización	37
--	----

Amenazas habituales	37
-------------------------------	----

21. Pruebas, diagnóstico y errores frecuentes **38**

Capas de pruebas	38
----------------------------	----

Lista de diagnóstico	38
--------------------------------	----

Problemas característicos	38
-------------------------------------	----

22. Rendimiento, accesibilidad y publicación **39**

Rendimiento medible	39
-------------------------------	----

Accesibilidad	39
-------------------------	----

Publicar	39
--------------------	----

Lista final de calidad	39
----------------------------------	----

23. Proyecto integrador guiado: Trivial **40**

Fases acumulativas	40
------------------------------	----

Criterios de evaluación	40
-----------------------------------	----

24. Glosario y referencias **41**

Glosario esencial	41
-----------------------------	----

Documentación de referencia	41
---------------------------------------	----

1. Mapa del manual y de los proyectos

La colección de ejemplos conduce desde páginas sencillas hasta aplicaciones que combinan interfaz Razor, persistencia, API externas, API propia y pruebas. Este capítulo sirve como mapa para saber qué estudiar y en qué orden.

Familias de problemas cubiertas

Tipo de proyecto	Conceptos principales	Capítulos
Listas, agenda, tareas y juegos de preguntas	formularios, CRUD, validación, filtros, paginación, JavaScript con debounce	5-13, 17
Pokémon, Rick y Morty, NASA, clima, recetas o videojuegos	HttpClient, DTO, JSON, caché, imágenes, audio, carruseles, errores de red	14-15, 17
Trivial paso a paso	EF Core, SQLite, relaciones, API de lectura, CORS, cliente estático, temas y confirmación de borrado	10-17
Aplicaciones autenticadas	Identity, autorización, antifalsificación, secretos y protección de datos	18
Aplicaciones en tiempo real	SignalR, hubs, estado compartido, frecuencia de actualización y DTO pequeños	16
Pruebas	xUnit, WebApplicationFactory y Playwright	19 y manuales específicos

Ruta de aprendizaje recomendada

1. Construye primero una página sin base de datos para dominar Razor, PageModel y handlers.
2. Añade un modelo validado y un formulario POST. Comprueba los errores de ModelState.
3. Incorpora EF Core y SQLite; implementa CRUD y consultas asíncronas.
4. Añade relaciones, búsqueda y paginación sin cargar datos innecesarios.
5. Consume una API externa mediante un cliente tipado y DTO pequeños.
6. Expón una API propia y crea un cliente estático solo cuando exista un objetivo didáctico claro.
7. Termina con seguridad, pruebas, publicación y observabilidad.

Principio rector

Cada página debe tener una responsabilidad reconocible. La vista decide cómo presentar; el PageModel coordina la petición; los servicios contienen operaciones reutilizables; EF Core se ocupa de persistir; los DTO delimitan los datos que atraviesan una frontera.

2. Web, HTTP y arquitectura de ASP.NET Core

Razor Pages genera HTML en el servidor. El navegador solicita una URL, ASP.NET Core atraviesa una canalización de middleware, selecciona un endpoint y ejecuta el handler de la página.

Petición y respuesta

- Una petición contiene método HTTP, URL, cabeceras y, a veces, un cuerpo.
- GET consulta una representación y debe ser seguro; POST envía datos y suele cambiar estado.
- La respuesta contiene un código de estado, cabeceras y un cuerpo: HTML, JSON, archivo u otro formato.
- Los códigos 2xx indican éxito; 3xx redirección; 4xx problema de la petición; 5xx error del servidor.

De la URL a la página

Elemento	Responsabilidad
Kestrel	Servidor HTTP que recibe las conexiones.
Middleware	Componentes ordenados que gestionan errores, HTTPS, ficheros estáticos, autenticación y autorización.
Enrutamiento	Relaciona la URL con el endpoint de Razor Pages, controlador o hub.
PageModel	Carga datos, valida entrada y decide el resultado de la petición.
Vista .cshtml	Combina HTML y sintaxis Razor para producir la respuesta.

Razor Pages, MVC, API y Blazor

Razor Pages está orientado a páginas renderizadas en el servidor y organiza juntos la vista y su modelo. MVC separa controladores y vistas y puede resultar apropiado cuando la aplicación se estructura por recursos o equipos. Los controladores de API devuelven datos, normalmente JSON. Blazor utiliza componentes interactivos. Un mismo proyecto ASP.NET Core puede combinar Razor Pages con controladores de API y SignalR.

No es Web Forms

Aunque ambos enfoques giran alrededor de una página, Razor Pages forma parte de ASP.NET Core, es comprobable, utiliza inyección de dependencias, routing moderno y HTML estándar; no reproduce el ciclo de vida ni el estado oculto de Web Forms.

3. Crear y conocer un proyecto .NET 10

SDK y comandos esenciales

Terminal

```
1 dotnet --info
2 dotnet new webapp -n Biblioteca
3 cd Biblioteca
4 dotnet restore
5 dotnet build
6 dotnet run
7 dotnet watch
```

dotnet watch recompila y reinicia cuando cambia el código. El fichero

Properties/launchSettings.json define perfiles y URL de desarrollo; no gobierna la publicación. La dirección efectiva también puede establecerse con **--urls** o con variables de entorno.

El archivo de proyecto

Biblioteca.csproj

```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2   <PropertyGroup>
3     <TargetFramework>net10.0</TargetFramework>
4     <Nullable>enable</Nullable>
5     <ImplicitUsings>enable</ImplicitUsings>
6   </PropertyGroup>
7 </Project>
```

- **Microsoft.NET.Sdk.Web** incorpora el SDK web y referencias compartidas de ASP.NET Core.
- **Nullable** obliga a razonar sobre la posible ausencia de valores.
- **ImplicitUsings** añade espacios de nombres habituales sin repetir using.
- Las referencias NuGet se expresan mediante **PackageReference**; una referencia a otro proyecto usa **ProjectReference**.

Estructura mínima

Ruta	Contenido
Program.cs	registro de servicios y canalización HTTP
Pages/	vistas .cshtml y PageModel .cshtml.cs
Pages/Shared/	layout, parciales y elementos compartidos
wwwroot/	CSS, JavaScript, imágenes y bibliotecas visibles desde el navegador
appsettings*.json	configuración por entorno
Properties/launchSettings.json	perfiles locales de ejecución

Actividades propuestas

1. Crea un proyecto net10.0, cambia sus URL de desarrollo y localiza la respuesta 404 al solicitar una ruta inexistente.
2. Añade una nueva página Ayuda y enlázala desde el pie mediante un Tag Helper.

4. Program.cs, servicios y middleware

Configuración mínima

Program.cs

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services.AddRazorPages();
4
5 var app = builder.Build();
6
7 if (!app.Environment.IsDevelopment())
8 {
9     app.UseExceptionHandler("/Error");
10    app.UseHsts();
11 }
12
13 app.UseHttpsRedirection();
14 app.UseStaticFiles();
15 app.UseRouting();
16 app.UseAuthorization();
17 app.MapRazorPages();
18 app.Run();
```

La primera mitad crea y configura el contenedor de servicios. La segunda construye la aplicación y define cómo se procesará cada petición. El orden de ciertos middleware importa: el enrutamiento debe conocer el endpoint antes de autorizarlo; la gestión global de excepciones debe envolver los componentes que pueden fallar.

Ciclos de vida de la inyección de dependencias

Registro	Duración	Uso habitual
AddTransient	nueva instancia en cada resolución	servicios ligeros y sin estado
AddScoped	una instancia por petición	DbContext y servicios de negocio ligados a la petición
AddSingleton	una instancia durante la aplicación	estado inmutable o servicios seguros para concurrencia

Error frecuente

No inyectes un servicio scoped, como DbContext, dentro de un singleton. El singleton viviría más que la dependencia y compartiría estado de una petición con otras.

Registrar servicios propios

CSHARP

```
1 builder.Services.AddScoped<IServicioPreguntas, ServicioPreguntas>();
2 builder.Services.AddMemoryCache();
3 builder.Services.AddHttpClient<ClientePokemon>(cliente =>
4 {
5     cliente.BaseAddress = new Uri("https://pokeapi.co/api/v2/");
6     cliente.Timeout = TimeSpan.FromSeconds(15);
7 });
```

5. Razor: HTML dinámico sin perder claridad

Directivas principales

Directiva	Uso
@page	convierte el .cshtml en una página enrutable; puede incluir plantilla de ruta
@model	declara el tipo de PageModel disponible mediante Model
@using	importa un espacio de nombres
@inject	inyecta un servicio en la vista; úsalo con moderación
@section	aporta contenido a una sección definida por el layout

Detalles.cshtml

```
1 @page "/productos/{id:int}"
2 @model DetallesModel
3
4 <h1>@Model.Producto.Nombre</h1>
5
6 @if (Model.Producto.EnOferta)
7 {
8     <span class="badge text-bg-success">Oferta</span>
9 }
10
11 <ul>
12 @foreach (var etiqueta in Model.Producto.Etiquetas)
13 {
14     <li>@etiqueta</li>
15 }
16 </ul>
```

Codificación automática

Razor codifica como HTML el contenido mostrado con **@expresión**. Así, un texto introducido por el usuario no se interpreta como etiquetas ni scripts. **Html.Raw** omite esa protección y solo debe recibir HTML de confianza y sanitizado. Para alumnado, la regla segura es no utilizarlo salvo que exista una necesidad demostrable.

Bloques de código y funciones

Un bloque **@{ ... }** prepara valores exclusivos de la presentación. Si aparecen consultas, acceso a red o reglas de negocio, trasládalos al PageModel o a un servicio. Una vista clara contiene decisiones de formato, no operaciones de infraestructura.

6. PageModel, handlers y resultados

Una pareja de archivos

Index.cshtml.cs

```
1 public class IndexModel : PageModel
2 {
3     public string Mensaje { get; private set; } = string.Empty;
4
5     public void OnGet()
6     {
7         Mensaje = $"Hora del servidor: {DateTime.Now:t}";
8     }
9 }
```

Index.cshtml

```
1 @page
2 @model IndexModel
3
4 <h1>Inicio</h1>
5 <p>@Model.Mensaje</p>
```

Convención de handlers

Método	Cuándo se ejecuta
OnGet / OnGetAsync	petición GET sin handler nombrado
OnPost / OnPostAsync	POST sin handler nombrado
OnPostEliminarAsync	POST con asp-page-handler=Eliminar
OnGetDescargarAsync	GET con handler Descargar

Resultados habituales

- **Page():** vuelve a renderizar la misma página; conserva ModelState y errores.
- **RedirectToPage():** responde con redirección y evita reenviar un formulario al recargar.
- **NotFound():** devuelve 404 cuando el recurso no existe.
- **BadRequest():** indica que la petición no es válida.
- **File():** devuelve un archivo o flujo.

CSHARP

```
1 public async Task<IActionResult> OnPostAsync()
2 {
3     if (!ModelState.IsValid)
4     {
5         return Page();
6     }
7
8     await _servicio.GuardarAsync(Entrada);
9     TempData["Exito"] = "Los cambios se han guardado.";
10    return RedirectToPage("./Index");
11 }
```

Patrón POST-Redirect-GET

Tras un POST correcto se redirige a un GET. Así el navegador no intenta repetir la operación si el usuario actualiza la página. Si hay errores, se devuelve Page() para mostrar la validación.

7. Routing, parámetros y generación de enlaces

Convención de carpetas

Pages/Index.cshtml responde a **/**; **Pages/Productos/Index.cshtml** a **/Productos**; **Pages/Productos/Editar.cshtml** a **/Productos/Editar**. Una plantilla en **@page** permite rutas más expresivas y restricciones de tipo.

HTML

```
1 @page "/preguntas/editar/{id:int:min(1)}"
```

Route values y query string

CSHARP

```
1 public async Task<IActionResult> OnGetAsync(int id, string? buscar)
2 {
3     var pregunta = await _context.Preguntas.FindAsync(id);
4     if (pregunta is null)
5     {
6         return NotFound();
7     }
8
9     Buscar = buscar;
10    Pregunta = pregunta;
11    return Page();
12 }
```

Tag Helpers para no escribir URL a mano

HTML

```
1 <a asp-page="./Editar" asp-route-id="@pregunta.Id"
2   class="btn btn-sm btn-outline-primary">
3     <i class="bi bi-pencil" aria-hidden="true"></i>
4     <span class="visually-hidden">Editar @pregunta.Texto</span>
5 </a>
6
7 <form method="get">
8     <input asp-for="Buscar" class="form-control">
9     <button class="btn btn-primary">Buscar</button>
10 </form>
```

Ventaja

Los Tag Helpers conocen el sistema de rutas y generan nombres, valores y token antifalsificación. Evitan enlaces frágiles y mantienen sincronizados el HTML y el modelo.

8. Model binding, formularios y validación

Modelo de entrada separado

CSHARP

```
1 public class CrearModel : PageModel
2 {
3     [BindProperty]
4     public EntradaPregunta Entrada { get; set; } = new();
5
6     public sealed class EntradaPregunta
7     {
8         [Required(ErrorMessage = "Escribe el enunciado.")]
9         [StringLength(300, MinimumLength = 10)]
10        public string Texto { get; set; } = string.Empty;
11
12        [Range(1, int.MaxValue, ErrorMessage = "Selecciona una categoría.")]
13        public int CategoriaId { get; set; }
14    }
15 }
```

Separar el modelo de entrada de la entidad de base de datos evita sobreasignación: el navegador solo puede enviar propiedades que el caso de uso permite modificar. Las anotaciones describen reglas compartidas por el binding, el servidor y los Tag Helpers de validación.

HTML

```
1 <form method="post">
2     <div asp-validation-summary="ModelOnly" class="text-danger"></div>
3
4     <div class="mb-3">
5         <label asp-for="Entrada.Texto" class="form-label"></label>
6         <textarea asp-for="Entrada.Texto" class="form-control"></textarea>
7         <span asp-validation-for="Entrada.Texto" class="text-danger"></span>
8     </div>
9
10    <button class="btn btn-primary" type="submit">Guardar</button>
11 </form>
12
13 @section Scripts {
14     <partial name="_ValidationScriptsPartial" />
15 }
```

ModelState

- Contiene los valores enlazados y los errores de conversión o validación.
- **ModelState.IsValid** solo debe consultarse en operaciones que reciben entrada.

- Puede añadirse un error de negocio con **ModelState.AddModelError**.
- Al devolver Page(), los Tag Helpers vuelven a mostrar los valores intentados y sus errores.

CSHARP

```
1  bool repetido = await _context.Categorias
2      .AnyAsync(c => c.Nombre == Entrada.Nombre);
3
4  if (repetido)
5  {
6      ModelState.AddModelError(
7          "Entrada.Nombre",
8          "Ya existe una categoría con ese nombre.");
9      return Page();
10 }
```

Validación y seguridad

La validación del navegador mejora la experiencia, pero puede omitirse. La aplicación siempre debe validar otra vez en el servidor y la base de datos debe proteger restricciones críticas con índices únicos o claves.

9. Interfaz compartida: layout, parciales y estado efímero

_Layout.cshtml

El layout contiene la estructura común: meta viewport, hojas de estilo, barra de navegación, **@RenderBody()**, pie y scripts. **@RenderSection("Scripts", required: false)** permite que una página añada JavaScript después de las bibliotecas comunes.

HTML

```
1 <!doctype html>
2 <html lang="es" data-bs-theme="light">
3 <head>
4     <meta charset="utf-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>@ViewData["Title"] - Trivial</title>
7     <link id="temaCss" rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css">
8     <link rel="stylesheet" href="~/lib/bootstrap-icons/font/bootstrap-icons.min.css">
9 </head>
10 <body class="d-flex flex-column min-vh-100">
11     <partial name="_Navegacion" />
12     <main class="container py-4 flex-grow-1">@RenderBody()</main>
13     <partial name="_Pie" />
14     <script src="~/lib/bootstrap/js/bootstrap.bundle.min.js"></script>
15     <script src="~/js/temas.js"></script>
16     @await RenderSectionAsync("Scripts", required: false)
17 </body>
18 </html>
```

_ViewImports y _ViewStart

_ViewImports.cshtml comparte @using, @namespace y Tag Helpers. **_ViewStart.cshtml** selecciona el layout para una rama de páginas. Ambos se aplican jerárquicamente por carpetas.

ViewData, ViewBag y TempData

Mecanismo	Duración	Uso recomendado
Propiedad del PageModel	petición actual	datos principales, tipados
ViewData	petición actual	título y metadatos pequeños
TempData	hasta la siguiente lectura	mensaje tras una redirección
Session	varias peticiones	solo si existe un requisito real de sesión

10. Configuración, secretos y logging

Configuración por capas

ASP.NET Core combina appsettings.json, appsettings del entorno, secretos de desarrollo, variables de entorno y argumentos de línea de comandos. Las fuentes posteriores pueden sobrescribir las anteriores. No guardes claves reales en el repositorio.

appsettings.json

```
1 {
2   "ConnectionStrings": {
3     "Trivial": "Data Source=trivial.db"
4   },
5   "Apis": {
6     "Nasa": {
7       "BaseUrl": "https://api.nasa.gov/"
8     }
9   },
10  "Logging": {
11    "LogLevel": {
12      "Default": "Information",
13      "Microsoft.AspNetCore": "Warning"
14    }
15  }
16 }
```

BASH

```
1 dotnet user-secrets init
2 dotnet user-secrets set "Apis:Nasa:ApiKey" "TU_CLAVE"
```

Options pattern

CSHARP

```
1 builder.Services
2     .AddOptions<NasaOpciones>()
3     .BindConfiguration("Apis:Nasa")
4     .ValidateDataAnnotations()
5     .ValidateOnStart();
6
7 public sealed class NasaOpciones
8 {
9     [Required, Url]
10    public string BaseUrl { get; init; } = string.Empty;
11
12    [Required]
13    public string ApiKey { get; init; } = string.Empty;
14 }
```

Logging estructurado

CSHARP

```
1 _logger.LogInformation(
2     "Consultando {Cantidad} preguntas de la categoría {CategoriaId}",
3     cantidad,
4     categoriaId);
5
6 _logger.LogWarning(ex,
7     "La API externa no respondió para {Recurso}", recurso);
```

Usa plantillas con propiedades en lugar de interpolar cadenas: el proveedor de logs puede filtrar y analizar cada campo. No registres contraseñas, claves, cookies, tokens ni datos personales innecesarios.

11. Entity Framework Core 10 y SQLite

EF Core traduce consultas LINQ a SQL, realiza seguimiento de cambios y persiste entidades. SQLite facilita ejemplos autocontenidos, aunque sus particularidades no deben confundirse con las de un servidor de base de datos completo.

Paquetes y herramientas

BASH

```
1 dotnet add package Microsoft.EntityFrameworkCore.Sqlite
2 dotnet add package Microsoft.EntityFrameworkCore.Design
3 dotnet tool install --global dotnet-ef
```

Entidad y DbContext

CSHARP

```
1 public class Categoria
2 {
3     public int Id { get; set; }
4
5     [Required, StringLength(80)]
6     public string Nombre { get; set; } = string.Empty;
7
8     public List<Pregunta> Preguntas { get; set; } = [];
9 }
10
11 public class TrivialContext(DbContextOptions<TrivialContext> options)
12     : DbContext(options)
13 {
14     public DbSet<Categoria> Categorias => Set<Categoria>();
15     public DbSet<Pregunta> Preguntas => Set<Pregunta>();
16
17     protected override void OnModelCreating(ModelBuilder modelBuilder)
18     {
19         modelBuilder.Entity<Categoria>()
20             .HasIndex(c => c.Nombre)
21             .IsUnique();
22     }
23 }
```

Registro en Program.cs

```
1 builder.Services.AddDbContext<TrivialContext>(options =>
2     options.UseSqlite(
3         builder.Configuration.GetConnectionString("Trivial"));
```

Migraciones

BASH

```
1 dotnet ef migrations add Inicial
2 dotnet ef database update
3 dotnet ef migrations list
4 dotnet ef migrations script
```

Una migración describe el paso de un esquema a otro y debe revisarse antes de aplicarla. El archivo .db no sustituye las migraciones en un equipo. En producción conviene generar y revisar un script o emplear un proceso de despliegue controlado.

Seguimiento y consultas de lectura

Las consultas que solo muestran datos pueden usar **AsNoTracking()**. Evita materializar pronto con `ToListAsync` si todavía puedes filtrar, ordenar, proyectar o paginar en SQL.

Regla de rendimiento

Construye primero un `IQueryable`; añade `Where`, `OrderBy`, `Select`, `Skip` y `Take`; ejecuta al final con `ToListAsync`. Cargar 1.000 filas para contarlas o agruparlas en memoria desperdicia tiempo y memoria.

12. CRUD asíncrono y relaciones

Listar con proyección

CSHARP

```
1 public sealed record CategoriaListaDto(  
2     int Id,  
3     string Nombre,  
4     int NumeroPreguntas);  
5  
6 public async Task OnGetAsync()  
7 {  
8     Categorias = await _context.Categorias  
9         .AsNoTracking()  
10        .OrderBy(c => c.Nombre)  
11        .Select(c => new CategoriaListaDto(  
12            c.Id,  
13            c.Nombre,  
14            c.Preguntas.Count()))  
15        .ToListAsync();  
16 }
```

Crear

CSHARP

```
1 public async Task<IActionResult> OnPostAsync()  
2 {  
3     if (!ModelState.IsValid)  
4     {  
5         return Page();  
6     }  
7  
8     var categoria = new Categoria { Nombre = Entrada.Nombre.Trim() };  
9     _context.Categorias.Add(categoria);  
10    await _context.SaveChangesAsync();  
11  
12    TempData["Exito"] = "Categoría creada.";  
13    return RedirectToPage("./Index");  
14 }
```

Editar sin sobreasignación

CSHARP

```
1 var categoria = await _context.Categorias.FindAsync(Entrada.Id);
2 if (categoria is null)
3 {
4     return NotFound();
5 }
6
7 categoria.Nombre = Entrada.Nombre.Trim();
8 await _context.SaveChangesAsync();
```

Borrar y controlar relaciones

CSHARP

```
1 var categoria = await _context.Categorias
2     .Include(c => c.Preguntas)
3     .SingleOrDefaultAsync(c => c.Id == id);
4
5 if (categoria is null)
6 {
7     return NotFound();
8 }
9
10 if (categoria.Preguntas.Count != 0)
11 {
12     TempData["Error"] = "No se puede borrar una categoría con preguntas.";
13     return RedirectToPage("./Index");
14 }
15
16 _context.Categorias.Remove(categoria);
17 await _context.SaveChangesAsync();
```

Carga de datos relacionados

Técnica	Descripción	Cuándo
Include/ThenInclude	carga entidades relacionadas	cuando la vista necesita entidades completas
Select	proyecta solo columnas y agregados	listados y respuestas API
Explicit loading	carga una relación bajo demanda	casos puntuales y controlados
Lazy loading	acceso transparente	evitarlo en ejemplos iniciales por consultas ocultas

13. LINQ, búsqueda, normalización y paginación

Consulta componible

CSHARP

```
1 IQueryable<Pregunta> consulta = _context.Preguntas
2     .AsNoTracking()
3     .Include(p => p.Categoria);
4
5 if (categoriaId is > 0)
6 {
7     consulta = consulta.Where(p => p.CategoriaId == categoriaId);
8 }
9
10 if (!string.IsNullOrEmpty(buscar))
11 {
12     string texto = buscar.Trim();
13     consulta = consulta.Where(p => p.Texto.Contains(texto));
14 }
15
16 int total = await consulta.CountAsync();
17
18 Preguntas = await consulta
19     .OrderBy(p => p.Texto)
20     .Skip((pagina - 1) * tamanoPagina)
21     .Take(tamanoPagina)
22     .Select(p => new PreguntaListaDto(
23         p.Id, p.Texto, p.Categoria.Nombre))
24     .ToListAsync();
```

Búsqueda sin tildes ni diferencias de mayúsculas

La solución correcta depende de la base de datos y su colación. Convertir toda la tabla con una función C# puede impedir que EF traduzca la consulta o que use índices. En SQLite puede registrarse una colación o almacenarse una columna normalizada e indexada cuando el requisito es importante. Para conjuntos pequeños didácticos puede normalizarse, pero debe explicarse el coste.

CSHARP

```
1 static string Normalizar(string texto)
2 {
3     string descompuesto = texto
4         .Normalize(NormalizationForm.FormD);
5
6     return string.Concat(descompuesto
7         .Where(c => CharUnicodeInfo.GetUnicodeCategory(c)
8             != UnicodeCategory.NonSpacingMark))
9         .ToUpperInvariant();
10 }
```

Modelo de paginación

- Valida pagina ≥ 1 y limita el tamaño máximo para impedir consultas enormes.
- Cuenta antes de aplicar Skip y Take.
- Conserva buscar, categoría y orden en los enlaces de páginas.
- Define siempre un OrderBy estable antes de paginar.
- En API públicas considera paginación por cursor para volúmenes grandes.

Actividades propuestas

1. Crea un listado de preguntas con categoría, texto y número de respuestas, sin materializar entidades completas.
2. Añade filtros combinables, ordenación ascendente/descendente y paginación que conserve el estado en la URL.
3. Compara el SQL generado al usar Select frente a Include y entidades completas.

14. Programación asíncrona en aplicaciones web

Por qué async/await

Las operaciones de base de datos, red y archivos esperan recursos externos. Durante esa espera, await devuelve el hilo al servidor para que pueda atender otra petición. No hace que una consulta individual sea mágicamente más rápida; mejora la capacidad de atender muchas peticiones sin bloquear hilos.

CSHARP

```
1 public async Task OnGetAsync(CancellationTok... cancellationToken)
2 {
3     Productos = await _context.Productos
4         .AsNoTracking()
5         .OrderBy(p => p.Nombre)
6         .ToListAsync(cancellationToken);
7 }
```

Buenas prácticas

- Propaga CancellationToken a EF Core y HttpClient cuando tenga sentido.
- No uses .Result, .Wait() ni GetAwaiter().GetResult() dentro del flujo web.
- No envuelvas I/O asíncrona en Task.Run.
- Evita async void salvo manejadores de eventos de interfaz; en servidor usa Task.
- No lances tareas sin observarlas: las excepciones y el ciclo de vida quedarían fuera de control.

Secuencial o paralelo

Si la segunda operación necesita el resultado de la primera, espera secuencialmente. Si dos consultas de red son independientes, Task.WhenAll puede reducir la latencia; no ejecute en paralelo varias operaciones sobre el mismo DbContext porque no es seguro para concurrencia.

15. Consumir API externas con HttpClient y DTO

Cliente tipado

Program.cs

```
1 builder.Services.AddHttpClient<ClienteRickMorty>(cliente =>
2 {
3     cliente.BaseAddress = new Uri("https://rickandmortyapi.com/api/");
4     cliente.DefaultRequestHeaders.UserAgent.ParseAdd("CursoRazor/1.0");
5     cliente.Timeout = TimeSpan.FromSeconds(15);
6 });
```

CSHARP

```
1 public sealed class ClienteRickMorty(
2     HttpClient http,
3     ILogger<ClienteRickMorty> logger)
4 {
5     public async Task<PersonajeDto?> ObtenerAsync(
6         int id,
7         CancellationToken cancellationToken)
8     {
9         try
10        {
11            return await http.GetFromJsonAsync<PersonajeDto>(
12                $"character/{id}", cancellationToken);
13        }
14        catch (HttpRequestException ex)
15        {
16            logger.LogWarning(ex,
17                "No se pudo obtener el personaje {PersonajeId}", id);
18            return null;
19        }
20    }
21 }
```

DTO: un contrato pequeño

CSHARP

```
1 public sealed record PersonajeDto(
2     int Id,
3     string Name,
4     string Status,
5     string Species,
6     string Image);
```

Un DTO representa el JSON que se intercambia, no el modelo de base de datos ni necesariamente todo el documento remoto. Reduce acoplamiento, evita transportar propiedades inútiles y permite adaptar nombres o estructuras. Con System.Text.Json, el tratamiento habitual de mayúsculas y minúsculas en ASP.NET Core suele ser flexible, pero los nombres especiales pueden mapearse con `JsonPropertyName`.

Estados de interfaz

Estado	Respuesta de la página
Cargando	en Razor Pages la carga ocurre antes del HTML; puede mostrarse un spinner solo para operaciones posteriores con JS
Sin resultados	mensaje explícito y posibilidad de retirar filtros
404 remoto	distinguir recurso inexistente de caída de servicio
Timeout o red	mensaje recuperable y log técnico
JSON inválido	registrar contexto sin exponer datos sensibles

IMemoryCache

CSHARP

```

1 public async Task<IReadOnlyList<PokemonResumenDto>> ListarAsync(
2     CancellationToken cancellationToken)
3 {
4     return await cache.GetOrCreateAsync("pokemon:listado", async entrada =>
5     {
6         entrada.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(20);
7         entrada.SlidingExpiration = TimeSpan.FromMinutes(5);
8
9         return await http.GetFromJsonAsync<ListaPokemonDto>(
10             "pokemon?limit=200&offset=0", cancellationToken)
11             is { } respuesta
12             ? respuesta.Results
13             : [];
14     }) ?? [];
15 }

```

Caché no es almacenamiento

La memoria se pierde al reiniciar y no se comparte entre servidores. No caches errores indefinidamente ni respuestas personalizadas de un usuario bajo una clave común. Define expiración y tamaño según el coste y la volatilidad de los datos.

16. API propia, controladores, JSON y CORS

Razor Pages y controladores en el mismo proyecto

CSHARP

```
1 builder.Services.AddRazorPages();
2 builder.Services.AddControllers();
3
4 // ... middleware ...
5 app.MapRazorPages();
6 app.MapControllers();
```

CSHARP

```
1 [ApiController]
2 [Route("api/preguntas")]
3 public class PreguntasController(TrivialContext context) : ControllerBase
4 {
5     [HttpGet]
6     public async Task<ActionResult<IReadOnlyList<PreguntaApiDto>>> Get(
7         int? categoriaId,
8         int cantidad = 20,
9         CancellationToken cancellationToken = default)
10    {
11        cantidad = Math.Clamp(cantidad, 1, 100);
12
13        IQueryable<Pregunta> consulta = context.Preguntas.AsNoTracking();
14
15        if (categoriaId is > 0)
16        {
17            consulta = consulta.Where(p => p.CategoriaId == categoriaId);
18        }
19
20        return await consulta
21            .OrderBy(_ => EF.Functions.Random())
22            .Take(cantidad)
23            .Select(p => new PreguntaApiDto(
24                p.Id, p.Texto, p.RespuestaCorrecta))
25            .ToListAsync(cancellationToken);
26    }
27 }
```

CORS

La política del mismo origen pertenece al navegador. CORS solo es necesario cuando un cliente web cargado desde un origen distinto llama a la API. No es un mecanismo de autenticación y no protege una API de clientes que no son navegadores.

CSHARP

```
1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("ClienteWeb", policy =>
4         policy.WithOrigins("https://cliente.ejemplo.es")
5             .AllowAnyHeader()
6             .AllowAnyMethod());
7 });
8
9 // Después de routing y antes de autorización/endpoints.
10 app.UseCors("ClienteWeb");
```

Evita AllowAnyOrigin por comodidad

Permitir cualquier origen puede ser válido para una API pública de lectura, pero debe ser una decisión. Si se permiten credenciales, el origen debe ser explícito y la superficie de ataque requiere más revisión.

Cliente estático

JAVASCRIPT

```
1 const respuesta = await fetch("/api/preguntas?cantidad=10");
2
3 if (!respuesta.ok) {
4     throw new Error(`La API respondió ${respuesta.status}`);
5 }
6
7 const preguntas = await respuesta.json();
8 mostrarPreguntas(preguntas);
```

17. SignalR y estado en tiempo real

Cuándo usarlo

SignalR mantiene una conexión para que el servidor pueda enviar mensajes sin que el navegador pregunte continuamente. Es apropiado para chats, presencia, paneles en directo y juegos sencillos. Un CRUD tradicional no lo necesita.

CSHARP

```
1 builder.Services.AddSignalR();
2 app.MapHub<JuegoHub>("/hubs/juego");
3
4 public class JuegoHub(IGestorPartidas gestor) : Hub
5 {
6     public async Task EntrarEnPartida(string jugadorId)
7     {
8         if (!gestor.Existe(jugadorId))
9         {
10             throw new HubException("El jugador no pertenece al juego.");
11         }
12
13         await Groups.AddToGroupAsync(Context.ConnectionId, "partida-global");
14     }
15 }
```

Diseño de mensajes

- Envía DTO mínimos: si el navegador solo dibuja X e Y, no necesita velocidades ni propietario.
- No repitas datos estáticos como muros en cada actualización; envíalos al comenzar la ronda.
- Separa la frecuencia de simulación de la frecuencia de difusión del estado.
- Considera reconexión, identificadores caducados, salida explícita e inactividad.
- El estado compartido debe ser seguro para concurrencia; protege colecciones o usa estructuras adecuadas.

18. JavaScript pequeño: debounce, temas y almacenamiento local

Debounce de búsqueda

JAVASCRIPT

```
1 const buscador = document.querySelector("#Buscar");
2 let temporizador;
3
4 buscador?.addEventListener("input", () => {
5     clearTimeout(temporizador);
6
7     temporizador = setTimeout(() => {
8         buscador.form?.requestSubmit();
9     }, 500);
10 });
```

El debounce espera a que el usuario deje de escribir antes de enviar. GET hace que el filtro quede en la URL y pueda compartirse. Si la página se recarga en cada búsqueda, puede restaurarse el foco y colocar el cursor al final, aunque una alternativa mejor para filtros instantáneos pequeños es actualizar solo el listado.

Tema persistente

wwwroot/js/temas.js

```
1  const claveTema = "temaTrivial";
2  const temaGuardado = localStorage.getItem(claveTema) ?? "bootstrap-light";
3
4  function cambiarTema(tema) {
5      const esBootswatch = tema.startsWith("bootswatch-");
6      const nombre = tema.replace("bootswatch-", "");
7      const hojaTema = document.getElementById("temaCss");
8
9      hojaTema.href = esBootswatch
10         ? `https://cdn.jsdelivr.net/npm/bootswatch@5/dist/${nombre}/bootstrap.min.css`
11         : "/lib/bootstrap/css/bootstrap.min.css";
12
13     const oscuros = ["darkly", "cyborg", "slate", "solar", "superhero", "vapor"];
14     document.documentElement.dataset.bsTheme =
15         tema === "bootstrap-dark" || oscuros.includes(nombre)
16             ? "dark"
17             : "light";
18
19     localStorage.setItem(claveTema, tema);
20 }
21
22 cambiarTema(temaGuardado);
```

dataset

document.documentElement.dataset.bsTheme lee o escribe el atributo **data-bs-theme** del elemento html. Las propiedades dataset convierten nombres con guiones a camelCase: data-categoria-id se accede como dataset.categoriaId. Los valores siempre son cadenas.

Evitar lógica duplicada

Si el layout Razor y un cliente estático comparten tema, ambos deben cargar el mismo temas.js y usar el mismo identificador de enlace y clave de localStorage. Duplicar listas de temas en dos ficheros provoca divergencias y errores difíciles de detectar.

19. Bootstrap, iconos, Bootswatch y SweetAlert2

Dependencias locales con LibMan

libman.json

```
1 {
2   "version": "1.0",
3   "defaultProvider": "cdnjs",
4   "libraries": [
5     {
6       "library": "twitter-bootstrap@5",
7       "destination": "wwwroot/lib/bootstrap/"
8     },
9     {
10      "library": "bootstrap-icons@1",
11      "destination": "wwwroot/lib/bootstrap-icons/"
12    },
13    {
14      "library": "sweetalert2@11",
15      "destination": "wwwroot/lib/sweetalert2/"
16    }
17  ]
18 }
```

Mantener bibliotecas en wwwroot permite trabajar sin conexión y evita depender de un CDN durante clase. CDN simplifica una demostración, pero debe fijarse una versión y considerar disponibilidad, privacidad e integridad. El manual específico de Bootstrap explica el sistema visual; el de SweetAlert2 desarrolla las confirmaciones.

Confirmación de borrado reutilizable

JAVASCRIPT

```
1 document.addEventListener("submit", async event => {
2     const formulario = event.target.closest("form[data-confirmar-borrado]");
3     if (!formulario || formulario.dataset.confirmado === "true") return;
4
5     event.preventDefault();
6
7     const resultado = await Swal.fire({
8         icon: "warning",
9         title: "¿Eliminar el registro?",
10        text: "Esta acción no se puede deshacer.",
11        showCancelButton: true,
12        confirmButtonText: "Eliminar",
13        cancelButtonText: "Cancelar"
14    });
15
16    if (resultado.isConfirmed) {
17        formulario.dataset.confirmado = "true";
18        formulario.requestSubmit(event.submitter);
19    }
20 });
```

Accesibilidad de iconos

Un icono decorativo lleva `aria-hidden=true`. Si un botón solo contiene icono, necesita un nombre accesible mediante `aria-label` o texto `visually-hidden`. El atributo `title` por sí solo no basta.

20. Seguridad: antifalsificación, Identity y autorización

Protecciones por defecto

- Los formularios POST generados por Razor incluyen token antifalsificación y los handlers lo validan.
- Razor codifica la salida HTML de forma predeterminada.
- [BindProperty] hace explícitas las propiedades enlazadas.
- HTTPS, HSTS y cookies seguras protegen el transporte y la sesión cuando están configurados.

Autenticación y autorización

Autenticar responde quién es la persona; autorizar decide qué puede hacer. ASP.NET Core Identity gestiona usuarios, contraseñas, cookies, confirmación, recuperación y roles. Las políticas expresan requisitos mejor que dispersar comprobaciones manuales por handlers.

CSHARP

```
1 builder.Services.AddAuthorization(options =>
2 {
3     options.AddPolicy("AdministrarPreguntas", policy =>
4         policy.RequireRole("Administrador", "Profesor"));
5 });
6
7 builder.Services.AddRazorPages(options =>
8 {
9     options.Conventions.AuthorizeFolder(
10         "/Preguntas", "AdministrarPreguntas");
11 });
```

Amenazas habituales

Riesgo	Medida
SQL injection	consultas parametrizadas de EF Core; no concatenar SQL
XSS	codificación Razor; no introducir datos sin confianza en html de SweetAlert
CSRF	token antifalsificación en operaciones con cookie
Sobresignación	modelos de entrada o DTO explícitos
Claves filtradas	user-secrets en desarrollo y almacén seguro en producción
IDOR	comprobar autorización sobre el recurso, no solo que el id exista

21. Pruebas, diagnóstico y errores frecuentes

Capas de pruebas

Prueba	Qué verifica	Herramienta
Unitaria	reglas aisladas y PageModel con dependencias controladas	xUnit
Integración	pipeline, routing, serialización, EF y API	WebApplicationFactory + xUnit
Navegador	HTML, JavaScript, estilos y flujo real	Playwright

Lista de diagnóstico

- Lee la primera excepción propia del proyecto, no solo el último mensaje de la consola.
- Comprueba URL, puerto y entorno que realmente se están usando.
- Mira la pestaña Red del navegador para estado, URL y respuesta de fetch.
- Comprueba la consola para errores de JavaScript y rutas de ficheros estáticos.
- Activa logging de EF Core en desarrollo y examina el SQL de consultas problemáticas.
- Reproduce el error con el menor conjunto de pasos y datos posible.

Problemas característicos

Síntoma	Causa probable
No encuentra temas.js	ruta relativa incorrecta; usar ~/js/temas.js en layout o ruta absoluta /js/temas.js
Fondo claro con tema oscuro	elementos con bg-white/text-dark fijos o data-bs-theme no actualizado
SweetAlert no aparece	script no cargado, listener antes del DOM, botón navega antes de preventDefault
Selector incompleto	lista de Bootstrap mantenida a mano y desactualizada
Página /novedades falla	ruta/handler, null remoto, cambio de contrato o excepción no tratada
DbContext disposed	servicio con ciclo de vida incorrecto o tarea fuera de la petición

22. Rendimiento, accesibilidad y publicación

Rendimiento medible

- Proyecta DTO y pagina en la base de datos.
- Usa AsNoTracking en lecturas y evita N+1 consultas.
- Cachea datos externos estables con una clave y expiración correctas.
- No descargues 100.000 recursos para mostrar una página de 20.
- Comprime, redimensiona y carga de forma diferida las imágenes cuando proceda.
- Mide antes y después; optimizar sin evidencia puede empeorar la claridad sin beneficio.

Accesibilidad

- Usa HTML semántico, un h1 por vista y jerarquía de encabezados coherente.
- Cada control de formulario debe tener label asociado y errores comprensibles.
- Mantén foco visible, contraste suficiente y navegación por teclado.
- Las tablas de datos necesitan th y scope; en móvil ofrece desplazamiento con .table-responsive.
- No comuniques estado únicamente mediante color; añade texto o icono con nombre accesible.
- Respeta prefers-reduced-motion en animaciones propias.

Publicar

BASH

```
1 dotnet publish -c Release -o ./publicado
2 dotnet ./publicado/Biblioteca.dll
```

La publicación debe probarse en un entorno parecido al destino, con configuración de producción, base de datos y permisos reales. No publiques launchSettings.json como fuente de configuración. Configura proxies de confianza, HTTPS, logs, copia de seguridad y migraciones según el alojamiento.

Lista final de calidad

- Compila sin errores y revisa advertencias NuGet y de nulabilidad.
- Todos los POST validan entrada, autorización y recurso afectado.
- Los listados grandes filtran, proyectan y paginan en servidor.
- Los fallos externos muestran un estado útil y generan logs diagnósticos.
- Interfaz comprobada con teclado, móvil y temas claros/oscuros.
- Pruebas unitarias, integración y flujos críticos de navegador superados.

23. Proyecto integrador guiado: Trivial

Fases acumulativas

1. Página estática y layout común con navegación responsive.
2. Modelo Categoría, contexto SQLite, migración y listado.
3. CRUD de categorías con validación única y SweetAlert2 al borrar.
4. Modelo Pregunta relacionado, CRUD, selector de categoría y validación.
5. Búsqueda, filtros, ordenación y paginación conservando query string.
6. API de solo lectura con DTO, límites de cantidad y documentación de URL.
7. Cliente estático con fetch, estado de juego y explicación de respuestas correctas.
8. Selector Bootstrap/Bootswatch compartido por layout y cliente.
9. Pruebas xUnit de reglas y API; Playwright para CRUD, filtros, tema y borrado.
10. Publicación Release, configuración segura y revisión de accesibilidad.

Criterios de evaluación

Área	Evidencia
Arquitectura	PageModel delgados, servicios cuando hay reutilización, DTO en fronteras
Datos	migraciones, relaciones, restricciones y consultas eficientes
Web	routing, handlers, validación, PRG y antifalsificación
Interfaz	responsive, accesible, Bootstrap coherente y JS mínimo
Robustez	errores controlados, logs, cancelación y configuración segura
Pruebas	casos normales, límites, errores y flujos críticos

24. Glosario y referencias

Glosario esencial

Término	Definición breve
Binding	conversión de datos HTTP en parámetros y propiedades .NET
DTO	contrato de datos diseñado para atravesar una frontera
Endpoint	destino ejecutable seleccionado para una petición
Handler	método OnGet/OnPost que atiende una operación de página
Middleware	componente de la canalización HTTP
Migración	cambio versionado del esquema de base de datos
PageModel	clase que coordina la entrada y salida de una página
Proyección	Select que transforma entidades en la forma exacta requerida
Tag Helper	componente que enriquece HTML durante el renderizado del servidor
Tracking	seguimiento de entidades para detectar cambios en EF Core

Documentación de referencia

[Razor Pages: arquitectura y conceptos \(.NET 10\)](#)

[Tutorial de Razor Pages \(.NET 10\)](#)

[Sintaxis Razor](#)

[Razor Pages con EF Core](#)

[IHttpClientFactory](#)

[Pruebas de integración](#)

[Novedades de .NET 10](#)

[Novedades de EF Core 10](#)

Sobre versiones

Los ejemplos fijan net10.0 y evitan depender de una revisión concreta de bibliotecas de interfaz. Al instalar paquetes, revisa compatibilidad, avisos de seguridad y notas de versión; no copies números antiguos de un manual sin comprobarlos.